

LECTURE NOTES

Long Short-Term Memory Networks

A 30-minute introduction · companion to the slide deck and Jupyter notebook

At a glance

This lecture builds intuition for LSTMs in three steps. First we show why vanilla recurrent networks fail on long sequences. Then we introduce the LSTM cell as an additive memory protected by gates. Finally we work through two short PyTorch examples and discuss where LSTMs still earn their keep alongside Transformers.

Each section below corresponds to one or two slides and includes the relevant equations, a few sentences of speaking material, and pointers to the companion notebook. Suggested timings add up to 30 minutes plus a 2–3 minute buffer for questions.

1. Why sequences need memory

≈ 3 minutes · slides 2–3

A standard feed-forward network maps one input to one output and forgets everything else. That is fine for static problems — classifying an image, scoring a row of features — but it fails wherever meaning depends on context that arrived earlier. In language, the verb agrees with a noun ten words back. In speech, a phoneme is recognized only in the company of its neighbours. In time series, today's reading is informative only against the trajectory of the past hours or weeks.

A **recurrent neural network** (RNN) handles this by maintaining a hidden state $\mathbf{h}_t$ that is updated at every step from the previous state and the current input:

$$\mathbf{h}_t = \tanh(W_{hh} \mathbf{h}_{t-1} + W_{xh} \mathbf{x}_t)$$

This single hidden vector is meant to summarize everything seen so far — a compression of unbounded history into fixed dimensions. Conceptually elegant, but as we are about to see, optimizing such a network with backpropagation runs into a hard mathematical wall.

NOTE FOR THE LECTURER

Open with a concrete example. Read the sentence “*The keys, which Alice forgot on the kitchen table, ____*” out loud and ask the audience to predict the verb. The answer **are** depends on a fact (*keys* is plural) that arrived **nine words earlier**. This is the kind of dependency we want a model to learn.

2. The vanishing-gradient problem

≈ 5 minutes · slide 4

To train an RNN we backpropagate the loss through the unrolled computation. The chain rule produces a product of T Jacobians, one per time step. For the simple update above, each Jacobian factors into a diagonal matrix of $\tanh'$ values multiplied by the recurrent weight matrix W_{hh} :

$$\frac{\partial h_T}{\partial h_t} = \prod \text{diag}(\tanh') \cdot W_{hh} \quad ((T - t) \text{ factors})$$

This product is at the heart of why long-range learning is hard. Treat the recurrent factor as a linear map and let $\sigma_{\max}$ be its largest singular value. Then the operator norm of the product is bounded by $\sigma_{\max}^{T-t}$, which behaves exponentially in the number of steps:

- **If $\sigma_{\max} < 1$:** gradients shrink to zero exponentially as we look further back. Long-range signal is invisible to the optimizer. The model can only learn dependencies of a handful of steps. This is the **vanishing-gradient problem**.
- **If $\sigma_{\max} > 1$:** gradients explode. Updates become enormous, weights diverge, and training NaNs out. This is partially fixable by gradient clipping, but it is also a symptom of a deeper instability.

The companion notebook makes this concrete. We feed a length-100 zero sequence to a randomly initialized RNN cell, set $\tanh'(0) = 1$ to isolate the recurrent factor, and plot the gradient norm against time on a log scale. Both curves come out as straight lines — the unmistakable signature of an exponential.

KEY TAKEAWAY

Vanilla RNNs cannot learn dependencies that span more than 10–20 steps. Any architecture that hopes to handle long sequences must break the multiplicative structure of this gradient — that is exactly what LSTMs do.

3. The LSTM idea: an additive memory

≈ 4 minutes · slide 5

Hochreiter and Schmidhuber introduced LSTMs in 1997 with a single, decisive idea: alongside the hidden state, maintain a second state vector — the **cell state** c_t — that is updated **additively** rather than through a non-linearity. When nothing important happens, the cell state simply passes through; when something important does happen, a small increment is added. Three sigmoid **gates** decide what counts as important, what to write, and what to expose.

There are two states and three gates. That is the entire architecture:

- **Cell state** c_t — the long-term memory. Updated additively. Persists across many time steps.
- **Hidden state** h_t — the short-term, output-facing memory. Used by the next step's gates and by any downstream prediction head.
- **Forget gate** f_t — for each cell-state coordinate, how much of the old value to keep.
- **Input gate** i_t — how much of the new candidate value to write in.
- **Output gate** o_t — which parts of the cell state to expose at the output.

4. Anatomy of an LSTM cell

≈ 6 minutes · slides 6–9

At every time step the cell computes four affine transformations of the concatenation $[h_{t-1}, x_t]$, squashes three of them into $(0, 1)$ with the sigmoid, and squashes the fourth into $(-1, 1)$ with $\tanh$.

The full set of equations is:

$$\begin{aligned}
 f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\
 i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\
 \hat{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) \\
 c_t &= f_t \odot c_{t-1} + i_t \odot \hat{c}_t \\
 o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\
 h_t &= o_t \odot \tanh(c_t)
 \end{aligned}$$

The fourth line is the one that matters. It is the only line that is purely **additive** in the cell state, and it is the highway along which gradients can travel without being repeatedly squashed by a non-linearity.

4.1 The forget gate

The forget gate f_t is a sigmoid, so each of its components lies in $(0, 1)$. It acts coordinatewise on the previous cell state. A value near 1 means “keep this dimension untouched”; a value near 0 means “wipe it.” In practice the forget bias b_f is initialized to 1 (or even 2) so the network **starts out remembering everything** and only gradually learns when to forget. This single trick speeds up training dramatically.

4.2 The input gate and the candidate

The candidate update $\hat{c}_t$ — a tanh of an affine combination — proposes a vector of changes to the cell state. The input gate i_t then decides, dimension by dimension, how much of that proposal actually gets written. The product $i_t \odot \hat{c}_t$ is the actual increment. Splitting “what to write” from “how much” gives the network finer control than a single transformation could.

4.3 The output gate

The output gate o_t selects which parts of the cell state become visible at the output. The hidden state is $h_t = o_t \odot \tanh(c_t)$: the cell state is squashed to $(-1, 1)$ so it does not blow up the next layer’s inputs, then masked by the gate. The cell can hold many independent facts at once and the output gate exposes only the relevant ones at each step.

A CONCRETE EXAMPLE

While reading “*The keys, which Alice forgot, are...*” the cell state can quietly track three things in parallel: that the subject is plural, that we are inside a relative clause, and that the sentence is not yet complete. When the verb is to be predicted, the output gate exposes the “plural” feature, and the model picks **are** rather than **is**.

5. Why this fixes the gradient

≈ 3 minutes · slide 10

The gradient of the loss with respect to an early cell state c_t is again a product over time, but the dominant factor is no longer W_{hh} — it is the forget gate f_τ :

$$\frac{\partial c_T}{\partial c_t} \approx \prod f_\tau$$

If the forget gates stay near 1 along the path — as they typically do thanks to the initialization above — the product stays near 1, and the gradient survives across hundreds of steps. The exponential decay is replaced by a learned, mostly-identity recurrence. *This is the entire reason LSTMs work.*

A useful mental image: the cell state is a **conveyor belt** running along the top of the network. Information is dropped onto it (input gate), occasionally swept off (forget gate), and read from it (output gate). Gradients flow back along the belt almost unimpeded.

6. Training in practice

≈ 3 minutes · slide 11

LSTMs are not difficult to train, but a few practical settings matter. The defaults in PyTorch's `nn.LSTM` handle most of these automatically; if you implement the cell yourself, mind the following.

- **Initialize the forget bias to 1.** This biases the cell toward remembering everything at the start of training. The PyTorch source does this by default.
- **Clip gradients.** Even with gates, occasional spikes happen. Clip the global gradient norm to roughly 1.0 with `torch.nn.utils.clip_grad_norm_`.
- **Truncated backpropagation through time.** Don't unroll the entire sequence. Process chunks of, say, 64–256 steps and pass the final state to the next chunk as a detached tensor.
- **Use batch-first layout.** Set `batch_first=True`. The tensors then have the more familiar (B, T, D) shape; performance is unchanged.
- **Mind the learning rate.** Adam with 10^{-3} to 3×10^{-3} is a safe default. Larger rates combined with long sequences will diverge.
- **Apply dropout between layers, not inside the recurrence.** PyTorch's `dropout` argument on `nn.LSTM` applies dropout between stacked LSTM layers, which is the right place for it. Variational dropout (the same mask across time) is stronger but requires a custom implementation.

7. Common variants

≈ 2 minutes · slide 12

The 1997 LSTM has spawned a small ecosystem. The most common variants in 2026 are:

Variant	Year	Why it exists
GRU	2014	Two gates instead of three; merges cell and hidden state. Often matches LSTM with $\sim 25\%$ fewer parameters.
Bi-LSTM	2005	Two LSTMs running forward and backward over the sequence. The encoder of choice before Transformers — strong for tagging and named-entity recognition.
Stacked LSTM	—	Several LSTM layers in sequence. More capacity, but harder to optimize past 3–4 layers.
Peephole LSTM	2000	Gates also see the cell state itself. Helps with precise timing tasks; rarely used today.
ConvLSTM	2015	Replaces matrix multiplications with convolutions. Standard for spatio-temporal data such as video and radar.
xLSTM	2024	Modernised gating, exponential gates, matrix memory. Designed to compete with Transformers and Mamba at scale.

8. Worked examples

≈ 4 minutes · slides 13–14, notebook

The companion notebook contains two short examples designed to be run during or just after the lecture. Both are CPU-friendly and complete in well under a minute.

8.1 An LSTM cell from scratch in NumPy

The first example builds a single LSTM cell from elementary operations, with no autograd. Its purpose is mechanical clarity: the entire cell is fewer than ten lines.

```
def step(self, x, h_prev, c_prev):
    z = self.W @ np.concatenate([h_prev, x]) + self.b
    H = self.H
    f = sigmoid(z[0:H:1*H])
    i = sigmoid(z[1*H:2*H])
    g = np.tanh(z[2*H:3*H])      # candidate c-hat
    o = sigmoid(z[3*H:4*H])
    c = f * c_prev + i * g      # additive memory
    h = o * np.tanh(c)
    return h, c
```

Three sigmoids, one tanh, one elementwise multiply-and-add. Stacking the four gate matrices into a single W of shape $(4 \cdot H, \text{input} + H)$ lets us compute all four pre-activations in one matmul, which is exactly what production implementations do.

8.2 A real LSTM in PyTorch

The second example trains a small character-level model on a tiny corpus. After 600 steps of Adam the model produces text with real words, plausible spaces, and the occasional period — more than

any bigram model can do, and a clear sign that the LSTM is carrying state across many characters.

```
class CharLSTM(nn.Module):
    def __init__(self, vocab, embed=32, hidden=128):
        super().__init__()
        self.embed = nn.Embedding(vocab, embed)
        self.lstm = nn.LSTM(embed, hidden, batch_first=True)
        self.head = nn.Linear(hidden, vocab)

    def forward(self, x, state=None):
        e = self.embed(x)  # (B, T, embed)
        out, state = self.lstm(e, state)
        return self.head(out), state
```

The notebook finishes with a small time-series forecasting task — predicting the next value of a noisy sine wave from a window of 20 past values. With a single 32-unit LSTM and 80 epochs of training, the test-set predictions track the held-out signal closely. This is the pattern of problem at which LSTMs still excel: modest data, mostly-stationary dynamics, and an inductive bias toward sequential structure.

9. LSTMs versus Transformers in 2026

≈ 3 minutes · slide 15

Transformers have replaced LSTMs for large-scale language modelling, where parallel training and direct attention to any past token are decisive. But the picture is more nuanced for other workloads. Three properties keep LSTMs and their state-space descendants relevant:

- **Constant inference cost.** An LSTM's state is $O(d)$ per step regardless of how long the sequence has been running. A Transformer's KV cache grows with context, and its per-token compute is $O(T \cdot d)$.
- **Streaming friendliness.** LSTMs are naturally online: you feed in tokens one by one and read out predictions one by one. Transformers can be made to stream, but it takes engineering.
- **Tiny footprints.** A 2-layer, 64-hidden LSTM is a few hundred kilobytes and runs comfortably on a microcontroller. For wake-word detection, sensor anomaly detection, or any modest time-series task, it is usually the right choice.

Modern state-space models — xLSTM, Mamba, S4 — are direct conceptual descendants of the LSTM. They keep the linear-recurrence structure that makes LSTMs efficient at inference and add either a learned, parallelisable scan or matrix-valued memory to recover the modelling power of attention. Understanding LSTMs is therefore not a historical exercise; it is the foundation for understanding the architectures that are most likely to challenge Transformers next.

10. Takeaways

≈ 1 minute · slide 16

- **Vanilla RNNs fail on long sequences** because gradients are products of T Jacobians, which decay or explode exponentially in T .

- **LSTMs add an additive memory highway.** The cell state c_t carries information through time, and three sigmoid gates control what to forget, what to write, and what to read.
- **The forget gate is the star of the show.** When it stays near 1, the cell state passes through almost unchanged, and gradients survive long spans.
- **LSTMs still matter in 2026** for streaming inference, edge devices, and modest-scale time-series problems — and they remain the conceptual root of every modern state-space model.

FURTHER READING

Hochreiter & Schmidhuber (1997). Long Short-Term Memory. *Neural Computation* 9(8). The original paper.

Greff et al. (2017). LSTM: A Search Space Odyssey. *IEEE TNNLS*. Empirical study of LSTM variants — concludes that the vanilla 1997 design is hard to beat.

Olah (2015). Understanding LSTM Networks. The classic blog post; the diagrammatic style on slide 6 is borrowed from it.

Beck et al. (2024). xLSTM: Extended Long Short-Term Memory. The most recent attempt to scale LSTMs to compete with Transformers.